



Film & Kino Agent

Sitzung 03 – RAG

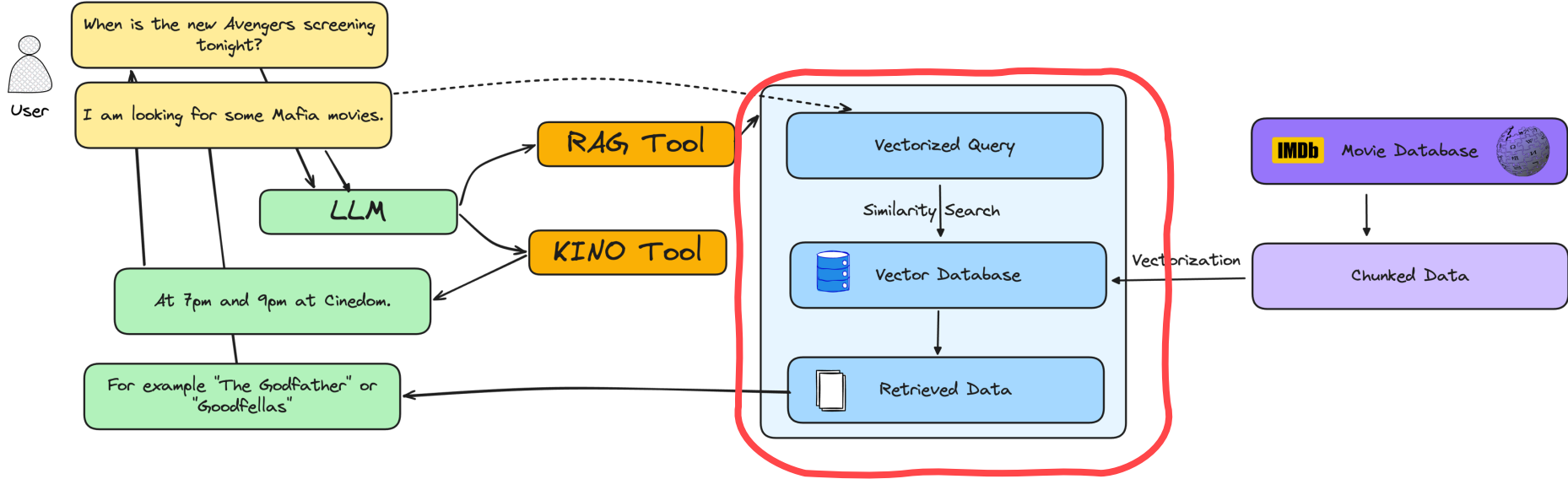
Prof. Dr. Richard Sieg
SoSe 2026

DIS22a

Termine

Datum	Inhalt
15.4	Kick-Off, Projektvorstellung, Setup
22.4	UI bauen, LLM-API-Anbindung, Prompt Engineering
29.4	Embeddings & Vektordatenbanken
6.5	Filmdatenbank vektorisieren
13.5	<i>Home Office</i>
20.5	RAG: Theorie + Implementation (Projektwoche)
27.5	<i>Home Office</i>
3.6	Tool Use & Agenten
10.6	Agenten-Implementation + Theorie: Evaluation
17.6	Gegenseitiger Nutzertest + Feinschliff
24.6	Feinschliff, Bugfixes
1.7	Abschlusspräsentationen

Architektur



Agenda

01.

RAG

02.

Fortgeschrittene Techniken

03.

RAG Evaluation

01.

RAG

RAG Konzept

R – Retrieval:

- Relevante Dokumente aus einer Quelle abrufen (Vektordatenbank, API, etc.)

A – Augmentation:

- Die abgerufenen Dokumente als Kontext zusammen mit der User-Anfrage dem LLM übergeben.

G – Generation:

- Das LLM generiert eine Antwort basierend auf dem kombinierten Input.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{1,†}, Ethan Perez^{*},

Aleksandra Piktus¹, Fabio Petroni¹, Vladimir Karpukhin¹, Naman Goyal¹, Heinrich Küttler¹,

Mike Lewis¹, Wen-tau Yih¹, Tim Rocktäschel^{1,†}, Sebastian Riedel^{1,†}, Douwe Kiela[‡]

[‡]Facebook AI Research; ¹University College London; ^{*}New York University;
plewis@fb.com

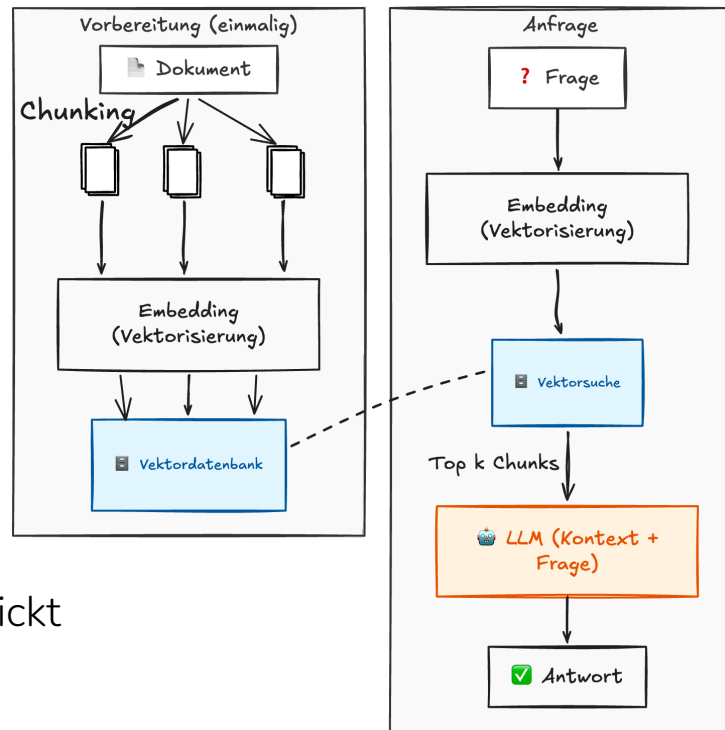
Die RAG Pipeline im Detail

Indexierung (offline, einmalig)

1. Dokumente werden in Abschnitte geteilt (Chunking)
2. Jeder Abschnitt wird in einen Embedding-Vektor umgewandelt
3. Die Vektoren werden zusammen mit Metadaten in einer Datenbank gespeichert

Abfrage (online, bei jeder Anfrage)

4. Die Frage wird ebenfalls in einen Vektor umgewandelt
5. Die ähnlichsten Abschnitte werden aus der Datenbank geholt
6. Frage + Abschnitte werden als Prompt an das LLM geschickt
7. Das LLM generiert eine Antwort basierend auf den Abschnitten



Chunking: Wann und warum?

Wann braucht man Chunking?

- Lange Dokumente (Papers, Handbücher, Berichte) die nicht komplett ins Context Window passen
- Dokumente mit mehreren Themen, wo nur ein Teil relevant ist
- Große Textmengen, in denen die Suche präziser sein soll

Wann braucht man kein Chunking?

- Kurze, fokussierte Texte (FAQs, Produktbeschreibungen)
- Strukturierte Daten mit einzelnen Einträgen
- Filmbeschreibungen sind ~100 Wörter lang → kein Chunking nötig.

Chunking Strategien

Fixed-Size Chunking:

- Text in gleichgroße Stücke aufteilen (z.B. 200 Tokens).
- Einfach, aber Sätze und Ideen werden mitten im Gedanken abgeschnitten.

Recursive Character Splitting:

- Versucht an natürlichen Grenzen zu trennen: erst Absätze, dann Sätze, dann Wörter.
- Guter Kompromiss aus Einfachheit und Qualität. Standard in vielen Frameworks.

Semantic Chunking:

- Nutzt Embeddings um zu erkennen, wo sich das Thema ändert. Trennt an Bedeutungsgrenzen.
- Qualitativ am besten, aber rechenintensiver (jeder Satz muss eingebettet werden).

Wichtig: Overlap

- Chunks sollten sich überlappen (10-20%), damit Kontext an den Grenzen nicht verloren geht.

Augmentation: Kontext in den Prompt einbauen

Du bist ein hilfreicher Film-Assistent. Beantworte Fragen ausschließlich basierend auf dem bereitgestellten Kontext. Wenn der Kontext die Frage nicht beantwortet, sage das ehrlich.

USER PROMPT

Kannst du mir Filme über Einsamkeit im Weltraum empfehlen?

CONTEXT

<item id="1">

Interstellar (2014): A team of explorers travel through a wormhole in space in an attempt to ensure humanity's survival.

Genres: Adventure, Drama, Sci-Fi. Rating: 8.6
</item>

<item id="2">

[2] Gravity (2013): Two astronauts work together to survive after an accident leaves them stranded in space.
Genres: Drama, Sci-Fi, Thriller. Rating: 7.7
</item>

<item id="3">

The Martian (2015): An astronaut becomes stranded on Mars and must find a way to survive.
Genres: Adventure, Drama, Sci-Fi. Rating: 8.0
</item>

- Kontext in XML tags mit Nummern oder Quellenangaben versehen (für Nachvollziehbarkeit)
- System Prompt: klar machen dass nur der Kontext genutzt werden soll (reduziert Halluzinationen)
- Metadaten (Genre, Rating, Jahr) mitgeben, damit das LLM filtern/sortieren kann

Generation: Antwort aus dem Kontext

Basierend auf den Filmen in meiner Datenbank kann ich dir folgende Empfehlungen geben:

1. **Interstellar** (2014, Rating 8.6) – Astronauten reisen durch ein Wurmloch um die Menschheit zu retten.
2. **The Martian** (2015, Rating 8.0) – Ein Astronaut ist allein auf dem Mars gestrandet.
3. **Gravity** (2013, Rating 7.7) – Zwei Astronauten kämpfen ums Überleben nach einem Unfall im All.

Alle drei Filme kombinieren Sci-Fi mit dem Thema Isolation im Weltraum.

- **Faithfulness:** Bezieht sich die Antwort nur auf den Kontext?
- **Relevance:** Beantwortet die Antwort die eigentliche Frage?
- **Grounding:** Sind Fakten nachvollziehbar aus dem Kontext ableitbar?

02.

Fortgeschrittene
Techniken

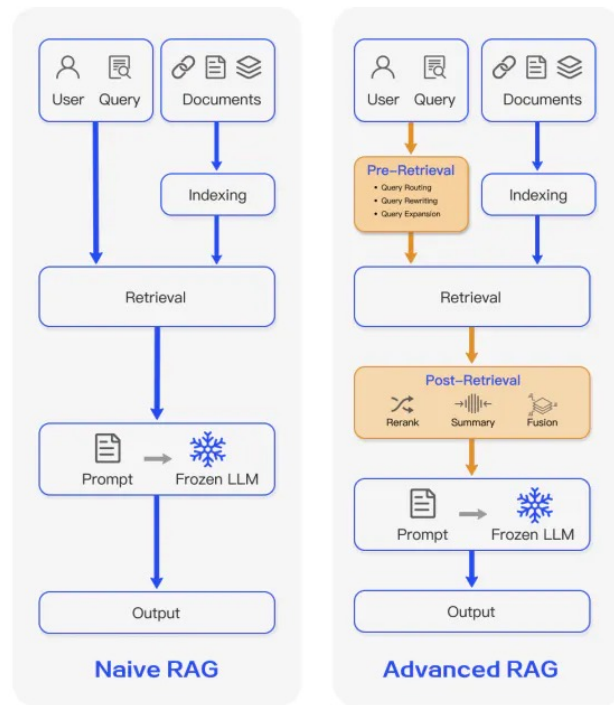
Advanced RAG: Stellschrauben (1)

Indexing:

- Verschiedene Chunking Strategien, Metadaten

Pre-Retrieval (Vor der Suche):

- Query Rewriting: LLM formuliert die Anfrage um, bevor gesucht wird
- HyDE: LLM generiert eine hypothetische Antwort, deren Embedding als Query genutzt wird (näher an Dokumenten als die ursprüngliche Frage)
- Step-Back Prompting: Abstrakte, übergeordnete Frage stellen statt der konkreten



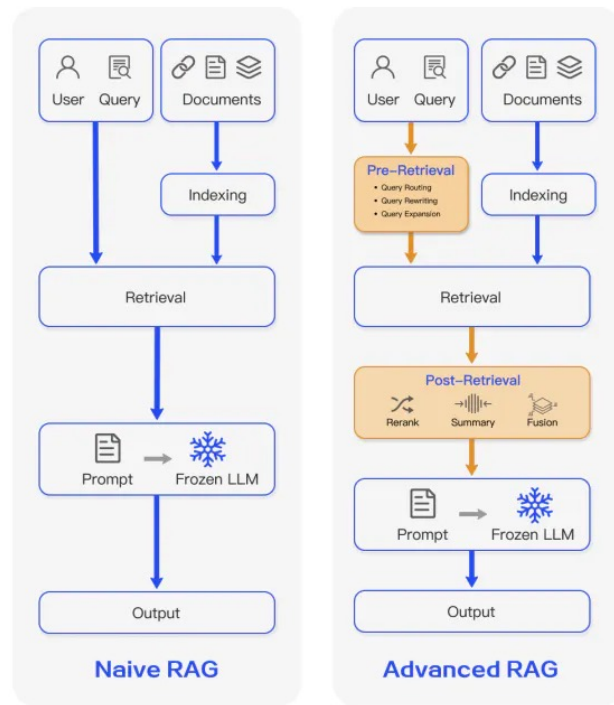
Advanced RAG: Stellschrauben (2)

Retrieval:

- Hybrid Search mit RRF (kennt ihr bereits)
- Metadaten-Filter (Genre, Jahr, Rating)
- Multi-Query: Mehrere Varianten der Frage parallel suchen

Post-Retrieval (Nach der Suche):

- **Reranking:** Cross-Encoder sortiert die Top-k Ergebnisse nach Relevanz neu
- **Compression:** Irrelevante Teile der Chunks zusammenfassen/entfernen
- **Contextual Retrieval:** Chunks mit Dokumentkontext anreichern bevor sie indiziert werden



HyDE: Mit einer hypothetischen Antwort suchen

User-Queries sind oft kurz und vage. Dokumente sind detailliert und lang.

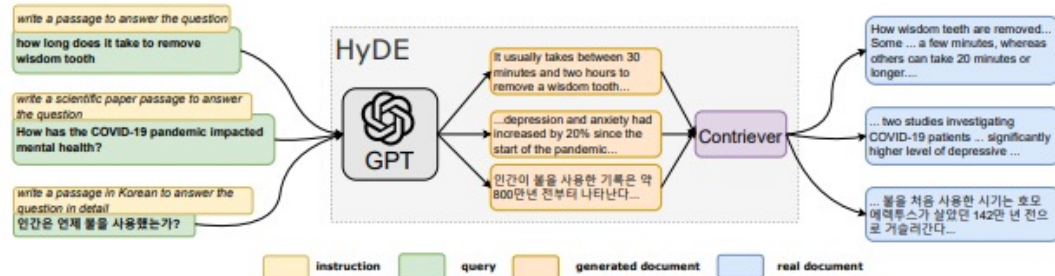
→ Die Embeddings von Query und Dokument liegen im Vektorraum weit auseinander.

Die Idee:

- LLM generiert eine hypothetische Antwort auf die Anfrage (muss nicht faktisch korrekt sein)
- Diese hypothetische Antwort wird eingebettet und als Query für die DB genutzt
- Das Embedding der hypothetischen Antwort ist viel näher an den echten Dokumenten

👍 Besserer Recall, weil die hypothetische Antwort strukturell den Dokumenten ähnelt.

👎 Extra LLM-Call, erhöht Latenz. Hypothetische Antwort kann in die falsche Richtung führen.



Reranking: Die zweite Stufe

Stage 1: Retrieval (schnell, breiter Fang)

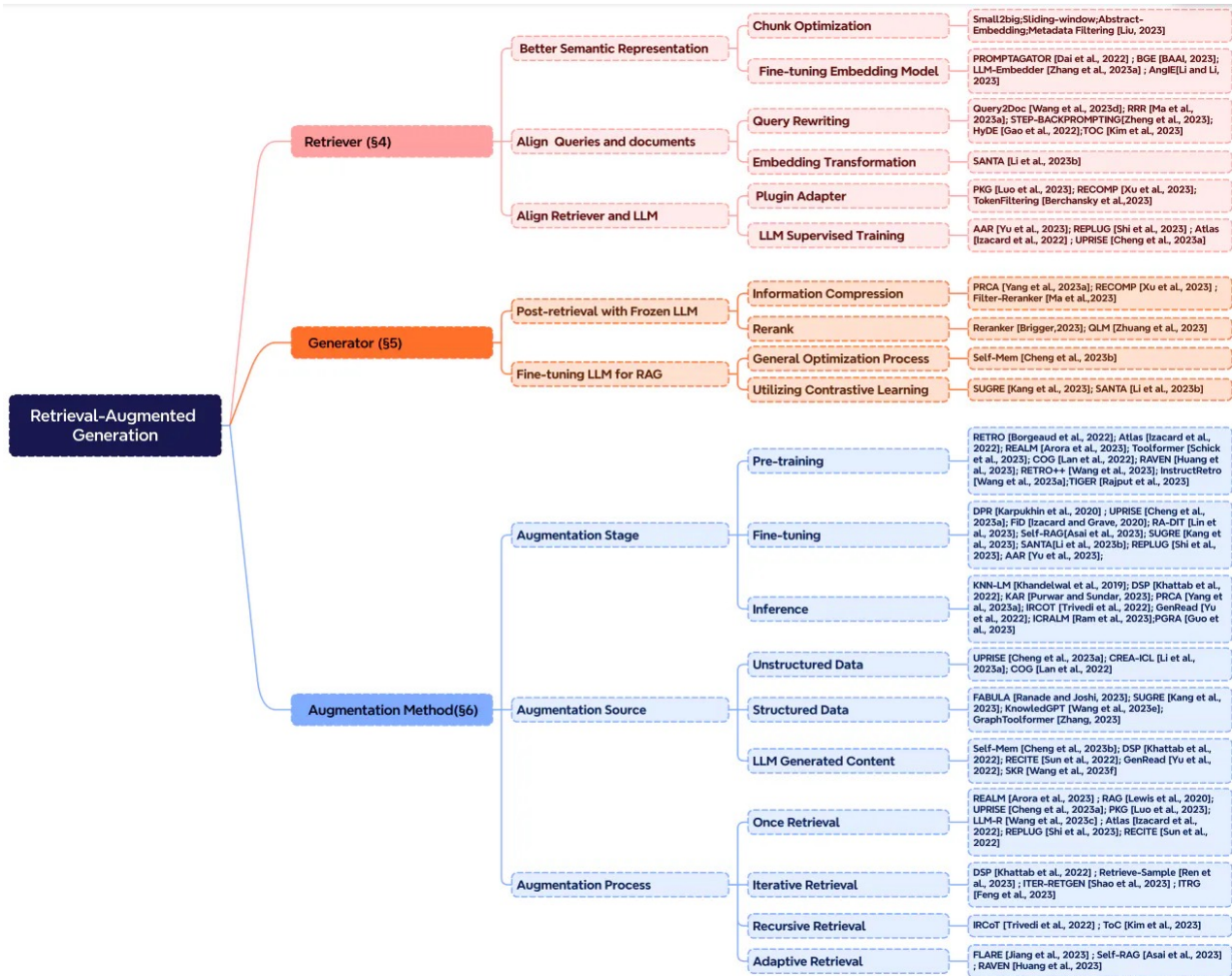
Bi-Encoder (Dense) und/oder BM25 (Sparse) → Top 50-100 Kandidaten

Stage 2: Reranking (langsam, präzise)

Cross-Encoder prüft jedes Query-Dokument-Paar einzeln → Top 5-10 finale Ergebnisse

Rückblick auf die Encoder-Typen aus letzter Sitzung:

- Bi-Encoder: schnell, Dokumente vorab berechnet → guter Retriever
- Cross-Encoder: langsam, aber sieht Query+Dokument zusammen → guter Reranker
- ColBERT: Mittelweg, kann beides



03.

RAG Evaluation

Wie messen wir die Qualität?

Retriever bewerten

- Findet er die richtigen Dokumente?
- **Context Precision:** Wie viele der abgerufenen Dokumente sind relevant? (Signal-to-Noise)
- **Context Recall:** Wurden alle relevanten Dokumente gefunden?

Generator bewerten

- Antwortet das LLM richtig?
- **Faithfulness:** Bezieht sich die Antwort nur auf den gegebenen Kontext? (keine Halluzinationen)
- **Answer Relevancy:** Beantwortet die Antwort tatsächlich die gestellte Frage